

LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBYEK

MODUL 6: POLYMORPHISM, METHOD OVERRIDING, OVERLOADING & DYNAMIC BINDING

1. Profil Proyek

Nama Proyek:	Sistem Manajemen Perpustakaan (Polymorphic Behavior & Dynamic Dispatch)
Nama Mahasiswa / NRP:	Arjuna Lanang Adiwersana / 3125522010
Program Studi / Kampus:	D3 PJJ Teknik Informatika - PENS PSDKU Sumenep
Fokus Pertemuan (P6):	Runtime Polymorphism (Overriding), Compile-time (Overloading), Upcasting, Polymorphic Collection & Dynamic Binding

2. Sprint Goal (P6)

Mengembangkan behavior objek pada proyek Sistem Manajemen Perpustakaan hasil P5 melalui method overriding, overloading, upcasting, dan polymorphic collection sehingga subclass (**Mahasiswa**, **Dosen**, **Tendik**) dapat merespons pemanggilan method yang sama (tampilkanPeran(), hitungDenda(), getMaximalPinjam()) dengan perilaku spesifik secara dinamis saat runtime (Dynamic Binding).

3. Sprint Backlog (P6)

ID	Sprint Backlog Item	Status
SB-01	Audit superclass (Anggota) dan subclass P5 (Mahasiswa, Dosen)	Done
SB-02	Menentukan behavior polymorphic yang dioverride (tampilkanPeran, hitungDenda, getMaximalPinjam)	Done
SB-03	Implementasi overriding pada subclass dengan anotasi @Override (termasuk subclass Tendik)	Done
SB-04	Implementasi method overloading pada Anggota (hitungDenda) dan Perpustakaan (cariBuku)	Done
SB-05	Implementasi upcasting reference superclass ke instance subclass	Done
SB-06	Membuat polymorphic collection (Anggota[]) dan iterasi polimorfik	Done
SB-07	Melakukan pengujian pembuktian dynamic method dispatch / dynamic binding	Done

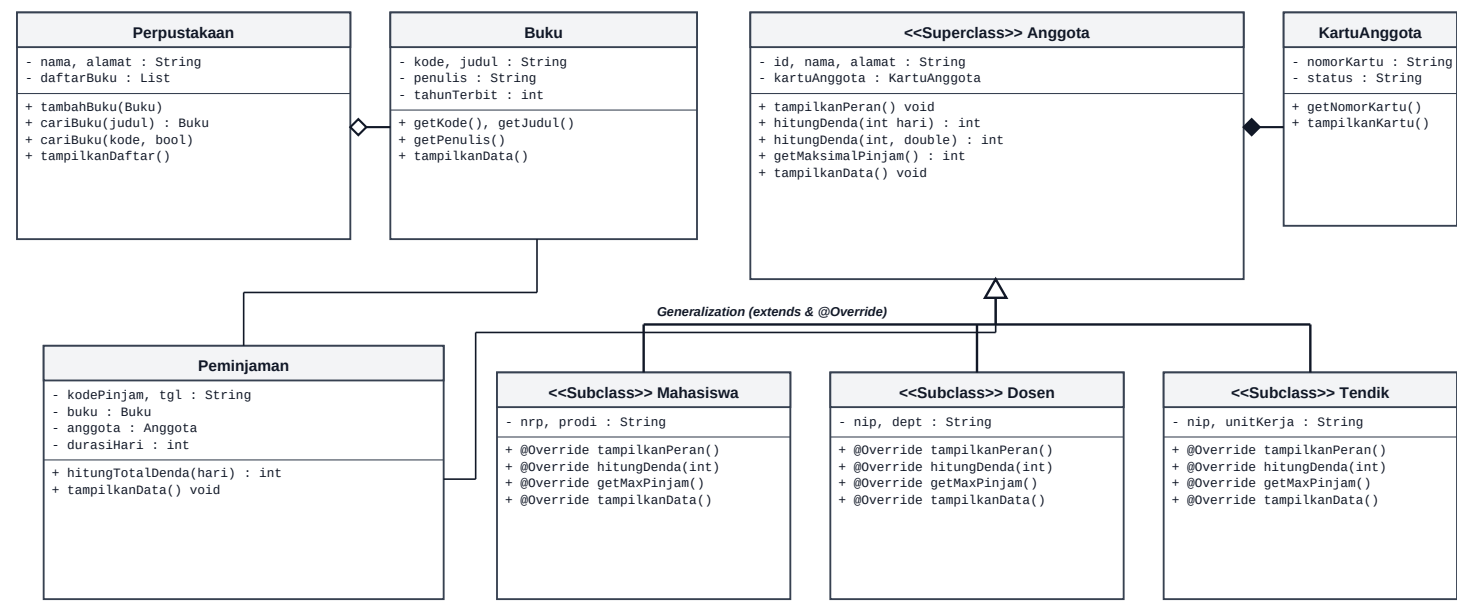
4. Bagian A: Hierarchy Class P5 & Behavior yang Dipilih untuk Polymorphism

Struktur hierarki P5 dikembangkan dengan menambahkan subclass Tendik untuk memperkaya variasi polimorfisme:

Superclass	Subclass	Method yang Di-override	Perilaku Spesifik (Polymorphic Behavior)
Anggota	Mahasiswa	tampilkanPeran() hitungDenda(int) getMaksimalPinjam()	- Peran: Akses koleksi skripsi & akademik - Denda terdiskon mahasiswa: Rp 500 / hari - Maksimal peminjaman: 3 buku
Anggota	Dosen	tampilkanPeran() hitungDenda(int) getMaksimalPinjam()	- Peran: Akses jurnal & referensi riset khusus - Hak istimewa akademik: Bebas denda (Rp 0) - Maksimal peminjaman: 5 buku
Anggota	Tendik	tampilkanPeran() hitungDenda(int) getMaksimalPinjam()	- Peran: Akses operasional kampus & administrasi - Denda tarif staf: Rp 750 / hari - Maksimal peminjaman: 4 buku

5. Bagian B: Class Diagram Terbaru (P6 - Polymorphism)

Diagram kelas menampilkan superclass, 3 subclass dengan method yang dioverride, overloading, dan relasi P4:



6. Contoh Overriding, Overloading, dan Penjelasan Upcasting

A. Contoh Method Overriding (@Override)	B. Contoh Method Overloading & Alasan
<pre>// Di Subclass Mahasiswa: @Override public int hitungDenda(int hari) { return hari * 500; // Tarif Mahasiswa } // Di Subclass Dosen: @Override public int hitungDenda(int hari) { return 0; // Bebas Denda }</pre>	<pre>// Di Class Anggota (Nama sama, parameter beda): public int hitungDenda(int hari) { ... } public int hitungDenda(int hari, double diskon) { ... } // Di Class Perpustakaan: public Buku cariBuku(String judul) { ... } public Buku cariBuku(String kode, boolean cetak) { ... }</pre> Alasan: Memudahkan pemanggil memilih variasi pencarian/kalkulasi tanpa menambah nama method baru.
C. Penjelasan Upcasting & Polymorphic Reference	D. Manfaat Arsitektural Polymorphism
Anggota a1 = new Mahasiswa(...); Anggota a2 = new Dosen(...); Anggota a3 = new Tendik(...); Upcasting menyimpan referensi objek subclass ke variabel bertipe superclass. Kode pemanggil cukup berinteraksi dengan tipe umum Anggota.	Kopling Rendah: Peminjaman dan Perpustakaan tidak bergantung pada subclass konkret. Open/Closed Principle: Menambah subclass baru (misal: AnggotaLuar) tidak mengubah logika peminjaman. Polymorphic Collection: Objek beragam dapat diproses dalam satu array loop.

7. Bukti Eksekusi Program (Console Output Main.java)

```
=== 1. UJI UPCASTING & POLYMORPHIC REFERENCE ===
[Sukses] Tiga objek subclass berhasil di-upcast ke tipe reference superclass 'Anggota'.
refMhs (Reference: Anggota) -> Objek: Mahasiswa | refDsn -> Dosen | refTdk -> Tendik

=== 2. UJI METHOD OVERRIDING & DYNAMIC BINDING ===
refMhs.tampilkanPeran() -> Peran : Mahasiswa (Akses Koleksi Skripsi & Akademik)
refDsn.tampilkanPeran() -> Peran : Dosen Pengajar & Peneliti (Akses Koleksi Jurnal)
refTdk.tampilkanPeran() -> Peran : Tenaga Kependidikan / Staf (Akses Operasional)
Denda (3 Hari Terlambat): Mahasiswa = Rp 1500 | Dosen = Rp 0 | Tendik = Rp 2250

=== 3. UJI METHOD OVERLOADING (COMPILE-TIME) ===
1. hitungDenda(4 hari) -> Rp 2000 | 2. hitungDenda(4 hari, diskon 50%) -> Rp 1000
1. cariBuku(judul) -> Ditemukan: Pemrograman Berorientasi Objek
2. cariBuku(kode, bool) -> [Info] Ditemukan buku: Struktur Data & Algoritma

=== 4. UJI POLYMORPHIC COLLECTION (ARRAY OF SUPERCLASS) ===
Iterasi pada 4 elemen array Anggota[] berhasil memanggil method spesifik tiap subclass secara dinamis.
=== 5. INTEGRASI TRANSAKSI PEMINJAMAN ===
PJ-001 (Mahasiswa) - Buku: PBO | Denda: Rp 1000 | PJ-002 (Dosen) - Buku: SDA | Denda: Rp 0
```

8. Tabel Pengujian Dynamic Binding & Penjelasan

Reference	Object Aktual	Method Dipanggil	Output yang Dihasilkan
Anggota	Mahasiswa	tampilkanPeran()	Peran : Mahasiswa (Akses Koleksi Skripsi & Akademik)
Anggota	Dosen	tampilkanPeran()	Peran : Dosen Pengajar & Peneliti (Akses Koleksi Jurnal)
Anggota	Tendik	tampilkanPeran()	Peran : Tenaga Kependidikan / Staf (Akses Operasional)
Anggota	Mahasiswa	hitungDenda(3)	Rp 1.500 (Tarif Mahasiswa: Rp 500 / hari)
Anggota	Dosen	hitungDenda(3)	Rp 0 (Hak Istimewa Bebas Denda)
Anggota	Tendik	hitungDenda(3)	Rp 2.250 (Tarif Tendik: Rp 750 / hari)

Mengapa method berbeda dieksekusi walau referensi sama? Pada Java, pemanggilan method non-static ditentukan saat runtime melalui **Dynamic Method Dispatch (Dynamic Binding)**. JVM menelusuri tabel metode virtual (vtable) milik objek aktual di heap memory (bukan tipe referensinya), sehingga implementasi subclass yang bersangkutan yang dieksekusi.

9. Sprint Review

Item Evaluasi	Hasil Evaluasi
Superclass & Subclass tersedia	Superclass Anggota serta Subclass Mahasiswa, Dosen, Tendik berfungsi optimal
Method overriding berhasil	Overriding tampilkanPeran(), hitungDenda(), getMaksimalPinjam(), tampilkanData() berhasil
Minimal 2 subclass memiliki behavior berbeda	3 subclass (Mahasiswa, Dosen, Tendik) memiliki logika denda & peran yang berbeda
Method overloading tersedia	Overloading pada Anggota.hitungDenda() dan Perpustakaan.cariBuku() berhasil
Upcasting & Polymorphic reference	Reference Anggota sukses menampung instance Mahasiswa, Dosen, dan Tendik
Polymorphic collection berhasil	Array Anggota[] menampung 4 objek subclass dan diiterasi secara polimorfik
Dynamic binding dapat dibuktikan	Terbukti pemanggilan method via reference superclass mengeksekusi method subclass aktual
Program berjalan & Kendala	Program terkompilasi dan berjalan 100% sukses tanpa kendala teknis

10. Sprint Retrospective

What Went Well? Polimorfisme dan dynamic binding membuat kode modular dan extensible. Relasi Peminjaman dapat menerima objek anggota apa pun tanpa perlu logika seleksi percabangan.

What Went Wrong? Perlu ketelitian membedakan *compile-time polymorphism* (overloading) dan *runtime polymorphism* (overriding).

Improvement: Pada modul P7 (Abstraction & Interface), superclass Anggota dapat didefinisikan sebagai abstract class dengan abstract method untuk memperjelas kontrak desain sistem.